ASSESSMENT TOOL 3 REPORT

NAME : MADHU BAVANA.B

Course Code: CSA1704
Course Name: Artificial Intelligence
Assessment Tool: Dry Run Test

*Question 1: Dry Run of A Search Algorithm**

Problem Statement

A graph consisting of nodes **A, B, C, D, E, and G** is provided, where each edge has an associated path cost and every node has a heuristic value. The task is to perform a dry run of the *A Search Algorithm** to find the optimal path from the start node **A** to the goal node **G**. During each iteration, the current node, Open List, Closed List, actual path cost **g(n)**, heuristic value **h(n)**, and evaluation function **f(n)=g(n)+h(n)** are recorded. Finally, the optimal path and total path cost are determined.

Aim

To determine the shortest path from node **A** to node **G** using the A* Search Algorithm.

Algorithm

1. Initialize the Open List with the start node.
2. Set the Closed List as empty.
3. Compute **f(n)=g(n)+h(n)**.
4. Select the node having the smallest **f(n)**.
5. Move the selected node to the Closed List.
6. Expand its neighboring nodes.
7. Update the path cost and parent information if a better path is found.
8. Repeat until the goal node is reached.
9. Display the optimal path and total cost.

Python Code

```
import heapq

graph = {
    'A': {'B':2,'C':4},
    'B': {'A':2,'C':3,'D':7,'E':2},
    'C': {'A':4,'B':3,'E':3},
    'D': {'B':7,'E':2,'G':2},
    'E': {'B':2,'C':3,'D':2},
    'G': {}
```

```
}

heuristic = {
    'A':7,
    'B':6,
    'C':4,
    'D':3,
    'E':2,
    'G':0
}

def astar(start, goal):

    open_list=[]
    heapq.heappush(open_list,(heuristic[start],start))

    g={node:float('inf') for node in graph}
    g[start]=0

    parent={start:None}
    closed=[]

    while open_list:

        f,current=heapq.heappop(open_list)

        if current in closed:
            continue

        closed.append(current)

        if current==goal:
            break

        for neighbour,cost in graph[current].items():

            new_g=g[current]+cost

            if new_g<g[neighbour]:
                g[neighbour]=new_g
                parent[neighbour]=current

                heapq.heappush(open_list,
                (new_g+heuristic[neighbour],neighbour))

    path=[]
    node=goal

    while node:
        path.append(node)
        node=parent[node]

    path.reverse()

    print("Optimal Path:",path)
    print("Total Cost:",g[goal])

astar('A','G')
```

Solution

The A* Search Algorithm starts from node **A** and evaluates each node using the function **f(n)=g(n)+h(n)**. Node **A** is expanded first, followed by **B**, because it provides the lowest evaluation cost. From node **B**, node **E** is reached with a lower path cost. Expanding **E** provides a better route to node **D**, reducing its total cost. Node **D** is then expanded, which directly reaches the goal node **G**. The search stops when the goal node is selected from the Open List. The algorithm successfully identifies the shortest path by considering both the actual path cost and the heuristic estimate.

Result

- **Optimal Path:** A → B → E → D → G
- **Total Path Cost: 8**

Question 2: Dry Run of Minimax with Alpha–Beta Pruning

Problem Statement

A game tree is given with a **MAX** node at the root and two **MIN** nodes as its children. The leaf nodes contain utility values. The objective is to perform a dry run of the **Minimax Algorithm with Alpha–Beta Pruning** by evaluating the tree from left to right. During execution, the values of **Alpha (α)**, **Beta (β)**, selected node values, and any pruned nodes are recorded. Finally, the best move for the MAX player, the final minimax value, and the pruned nodes are identified.

Aim

To determine the best move for the MAX player using the Minimax Algorithm with Alpha–Beta Pruning.

Algorithm

1. Start from the root MAX node.
2. Evaluate the left subtree first.
3. Update Alpha (α) and Beta (β) values.
4. Prune branches whenever **α ≥ β**.
5. Continue until all required nodes are evaluated.
6. Select the maximum value at the root.

Python Code

```
import math

tree=[
    [3,5,6],
    [9,1,2]
```

```
]

def minimax(node,alpha,beta,maximizing):

    if isinstance(node,int):
        return node

    if maximizing:

        value=-math.inf

        for child in node:
            value=max(value,minimax(child,alpha,beta,False))
            alpha=max(alpha,value)

            if alpha>=beta:
                break

        return value

    else:

        value=math.inf

        for child in node:
            value=min(value,minimax(child,alpha,beta,True))
            beta=min(beta,value)

            if alpha>=beta:
                break

        return value

answer=minimax(tree,-math.inf,math.inf,True)

print("Final Minimax Value =",answer)
```

Solution

The Minimax algorithm begins at the MAX node with $\alpha = -\infty$ and $\beta = +\infty$. The left MIN subtree is evaluated first, producing the minimum value **3**, which updates $\alpha = 3$ at the root. The right MIN subtree is then evaluated. After examining the values **9** and **1**, the Beta value becomes **1**. Since $\alpha (3) \geq \beta (1)$, the remaining leaf node with value **2** is pruned because it cannot influence the final decision. The MAX node compares the values returned by the two MIN nodes and selects the larger value **3**.

Result

- **Best Move for MAX:** Left Subtree
- **Final Minimax Value: 3**
- **Pruned Node: 2**

Conclusion

The assessment demonstrates the working of two important Artificial Intelligence algorithms. The *A Search Algorithm** efficiently finds the shortest path using path cost and heuristic information, while the **Minimax Algorithm with Alpha–Beta Pruning** determines the optimal decision in a game tree by eliminating unnecessary branches. Both algorithms improve computational efficiency and are widely used in pathfinding, robotics, and game-playing applications.